

10 — Outils & écosystème (annexe)

Table des matieres

[Vue d'ensemble](#)

[Python](#)

[pandas](#)

[numpy](#)

[scipy.stats](#)

[matplotlib](#)

[PyTorch](#)

[scikit-learn](#)

[git / conda](#)

[Pour aller plus loin](#)

Niveau : base → intermédiaire. **Prérequis** : aucun. Annexe transversale, à consulter au besoin. **Couvre la fiche source** : section 9 de concepts_FR.md.

Idée directrice. Cette annexe n'est **pas un tutoriel**. Pour chaque outil, on répond à une seule question : *pourquoi celui-ci pour ce besoin précis du projet ?* — et on signale les **pièges réellement rencontrés**. Pour apprendre la syntaxe, voir les documentations officielles ; ici, on explique les **choix** d'ingénierie d'iAlexMG.

Vue d'ensemble

Outil	Besoin du projet	Pourquoi celui-ci
Python	langage pivot	écosystème scientifique, lisibilité
pandas	manipulation des tirages	tables étiquetées, dates, export
numpy	calcul vectoriel & aléa	vitesse, RNG reproductible
scipy.stats	lois & tests	binom, χ^2 , géom, hypergeom, norm, beta, wilcoxon
matplotlib	figures « page de manuel »	contrôle fin du rendu

Outil	Besoin du projet	Pourquoi celui-ci
PyTorch	MLP & LSTM	autograd, nn.LSTM, GPU si besoin
scikit-learn	logistique & scaling	API fit/transform propre
git / conda	versionnage & environnement	reproductibilité

Python

Rôle. Langage unique du projet, du parsing HTML aux réseaux de neurones.

Pourquoi cet outil. L'écosystème scientifique (numpy/pandas/scipy/sklearn/torch) est en Python ; un seul langage couvre tout le pipeline, ce qui simplifie la reproductibilité.

Idiomes mobilisés. @dataclass(frozen=True) pour la GameConfig immuable (§8.4) ; @property pour les valeurs dérivées (ex. single_draw_probability) ; annotations de type pour la lisibilité ; pathlib pour des chemins portables.

Piège. L'immuabilité (frozen=True) protège la config, mais oblige à recréer l'objet pour le re-cibler — ce qui est précisément le comportement voulu.

pandas

Rôle. Charger, normaliser et explorer la table des tirages (date, n1..n6, bonus).

Pourquoi cet outil. Les **tables étiquetées** (colonnes nommées, index temporel), la gestion native des **dates** (.dt), les agrégations (groupby) et les **exports** (parquet/xlsx) en font l'outil naturel pour des données tabulaires datées.

Idiomes mobilisés. Méthodes **non in-place** (toujours réaffecter : df = df.sort_values(...)) ; nunique(axis=1) pour vérifier les 6 numéros distincts (§8.2) ; sort_values + reset_index ; accès .dt ; export to_parquet / to_excel.

Piège. Croire qu'une méthode modifie en place : la plupart **renvoient une copie** ; oublier de réaffecter laisse les données inchangées (bug silencieux).

numpy

Rôle. Calcul vectoriel (comptes, fréquences, simulations) et génération d'aléa.

Pourquoi cet outil. La **vectorisation** rend les simulations Monte-Carlo (§2.8) et les permutations (§2.9) rapides ; le générateur `default_rng` offre un aléa **reproductible** (seedé, §8.4).

Idiomes mobilisés. `np.sort(axis=1)` (tri par ligne, §8.1) ; `bincount` (comptes par numéro) ; `default_rng(seed)` (RNG moderne) ; `argpartition` (top-k pour `hit@6`, §5.7).

Piège. Utiliser l'ancien `np.random.seed` global plutôt que `default_rng` : moins reproductible et déconseillé ; préférer un générateur explicite passé en argument.

scipy.stats

Rôle. Fournir lois et tests exacts, cœur statistique des Phases 1-2.

Pourquoi cet outil. Implémentations fiables et testées des lois du document 02 et des tests du document 03 — inutile (et risqué) de les réécrire.

Correspondance avec les concepts. `binom` / `binomtest` (§1.1, test exact) ; `chi2` (§2.6) ; `geom` (§1.2) ; `hypergeom` (§1.3) ; `norm` (§1.4, z-test) ; `beta` (§1.6, posteriors et quantiles crédibles §3.3) ; `wilcoxon` (§5.8, test apparié).

Piège. Confondre test **exact** et **asymptotique** : pour de petits effectifs (séries T3), utiliser `binomtest` (exact) plutôt qu'un z asymptotique (§1.1).

matplotlib

Rôle. Produire les figures au gabarit « page de manuel scientifique » (fig01–fig11).

Pourquoi cet outil. Le **contrôle fin** du rendu permet le gabarit maison : titre centré, sous-sections **Hypothèse / Démarche / Résultats / Conclusion**, légendes numérotées, police serif.

Idiomes mobilisés. `rc_context` (style serif local sans polluer l'état global) ; `errorbar` (bandes de confiance T1-T3), `stem` (motifs n-grammes), `axhspan` (bandes) ; `savefig(dpi=...)` pour l'export net.

Piège. Modifier les `rcParams` globaux au lieu d'un `rc_context` local : effets de bord sur les figures suivantes.

PyTorch

Rôle. Implémenter le MLP (§5.11) et le LSTM (§5.13).

Pourquoi cet outil. L'**autograd** (différentiation automatique) et les briques toutes faites (nn.Module, nn.LSTM, BCEWithLogitsLoss, optimiseur Adam) rendent l'entraînement direct ; passage GPU possible si besoin.

Idiomes mobilisés. nn.Module (modèles) ; nn.LSTM (récurrence) ; BCEWithLogitsLoss (log-loss numériquement stable, §5.4) ; Adam (optimisation).

Pièges (Windows, réellement rencontrés).

-  **OpenMP** : définir KMP_DUPLICATE_LIB_OK=TRUE **avant** d'importer torch, sinon crash de duplication de librairie OpenMP.
 -  **Conflit setuptools** : incompatibilité entre torch (<82) et sip (≥75.8.1) ; épingler les versions dans l'environnement.
-

scikit-learn

Rôle. Régression logistique (§5.10) et standardisation des features (§5.2).

Pourquoi cet outil. L'API **fit / transform** impose proprement la discipline anti-fuite : StandardScaler.fit(train) puis .transform(test) (§5.2), garantissant que la normalisation n'apprend rien du futur.

Idiomes mobilisés. StandardScaler (standardisation apprise sur le train) ; LogisticRegression (baseline 3, coefficients interprétables).

Piège. Appeler fit sur l'ensemble complet (train + test) → **leakage par normalisation** (§5.1). Toujours fit sur le train seul.

git / conda

Rôle. Versionner le code et figer l'environnement.

Pourquoi ces outils. git trace l'historique (commits par étape, message préfixé Phase X TY:) et conda isole un environnement dédié reproductible.

Idiomes mobilisés. Commits par étape validée + push ; .gitignore (données traitées régénérables, §8.4) ; environnement conda dédié **ialexmg_649** ; environment.yml pour geler les versions.

Pièges.

- Versionner des **données lourdes régénérables** : préférer les gitignorer et garder la source brute (§8.4).
 - Se tromper d'environnement : le projet utilise **ialexmg_649** (⚠ pas un autre nom), sous **PowerShell**.
-

Pour aller plus loin

- **Documentations officielles** : pandas, numpy, scipy, matplotlib, scikit-learn, PyTorch — pour la syntaxe et les options détaillées.
- **Reproductibilité** : lockfiles, conteneurs (Docker) pour figer l'environnement au-delà de conda.
- **Lien §8.4** : ce document est le pendant « outillage » de la fiche reproductibilité. ``